EV SPARE PARTS APP

Developer Working Roadmap & Functional Requirements

Android only • Flutter/Dart • Version 1.0

Developer-ന് നേരിട്ട് നൽകാനുള്ള working specification. Budget quotation അല്ല. Requirements, workflows, architecture, data, security, testing, handover എന്നിവ ഉൾപ്പെടുന്നു.

Instructions to developer

മുഴുവൻ document review ചെയ്ത് ഓരോ item-നും Must / Should / Later priority, implementation approach, dependency, test case എന്നിവ രേഖപ്പെടുത്തുക. 07A.pdf-ൽ വ്യക്തമല്ലാത്ത feature behavior client എഴുതി അംഗീകരിക്കുന്നതുവരെ production logic ആയി കണക്കാക്കരുത്.

1. Project definition

Purpose: Android application for EV scooter spare-parts discovery, shopping, ordering and service support. Main business priority is spare-parts sales.

Platform: Android only. Entire app codebase must be Flutter/Dart. No website or iOS app in this scope.

Roles: Customer and Admin roles in one Flutter app with role-based screens. Admin permissions must be enforced by backend/database rules, not merely by hiding buttons.

Source: Use 07A.pdf for the additional Customer, Service Center and Spare Parts functions described in Section 8. Do not invent behavior where the PDF is unclear.

2. User experience and navigation

Dashboard: Provide clear entry points for Customer, Spare Parts and Service Center, matching the 07A.pdf main dashboard. Spare-parts shopping should be the primary journey.

Usability: Simple, consistent screens; minimal steps to find a part and place an order; readable text; accessible touch targets; support common Android phone sizes.

Screen states: Every data-driven screen must handle loading, empty, error, success and offline/poor-network states.

Language: Initial app language must be confirmed. Externalize UI strings so another language can be added later.

3. Customer module – MVP

Registration: Customer sign-up/sign-in, sign-out and session handling. Collect only necessary profile/contact details.

Vehicle profile: Allow customer to add EV scooter make/brand, model and variant/year if required. Exact fields to be confirmed by client.

Categories and catalogue: Browse active spare-parts categories and product listings.

Product details: Show name, images, part number/SKU, description, price, availability/stock and compatible scooter models. Do not claim compatibility unless Admin has entered it.

Search: Search by product name, part number/SKU, category and scooter make/model. Include no-result and retry states.

QR/Barcode scanner: Open camera scanner on request; scan QR and agreed retail barcode formats; resolve code to product/SKU; if no match, show not-found and allow manual search. Request camera permission only when needed.

Cart: Add/remove items and change quantity. Re-check price and stock at checkout. Prevent quantity above available stock unless backorders are approved.

Order submission: Collect required delivery/contact details, show item/charge summary, submit order and generate a unique, non-guessable Order ID.

Order history: Customer can view only their own orders and status history.

Contact: Provide Call + Visit options from O7A.pdf using contact/address details supplied by client.

4. Admin module – MVP

Admin access: Only approved accounts can access Admin functions. No public Admin self-registration.

Dashboard: Show new/pending orders and stock alerts based on database values.

Product management: Create, view, edit, archive/delete products. Fields: name, SKU/part number, description, category, price, stock, availability, compatible scooter models and images.

Images: Upload multiple images, reorder, replace and delete. Show progress and upload errors.

Categories: Create/edit/reorder/enable/disable categories.

Inventory: Update stock and availability. Record who changed stock and when.

Orders: View order list/details, verify payment, update order status and add internal notes.

Statuses: Suggested order states: Submitted/Pending, Confirmed, Processing, Shipped, Delivered, Cancelled. Validate allowed transitions.

Payment states: Track separately: Unpaid/Pending, Submitted/Under review, Verified, Rejected/Failed.

Audit trail: Record Admin identity and timestamp for important product, price, stock, order and payment changes.

5. Order and payment workflows

Manual UPI MVP: Customer submits order; app displays merchant UPI QR/instructions; customer pays and submits UTR/reference if required; Admin checks merchant account and marks payment verified. A screenshot alone is not proof of payment.

Automatic UPI: For automatic confirmation, use a payment provider that gives the merchant an approved API/webhook. Server verifies webhook signature, amount, currency, order reference and transaction status before marking payment verified.

Webhook safety: Handle duplicate events idempotently; store provider transaction reference; never trust client-side payment-success messages or client-supplied payment status.

Shop POS/UPI machine: Integrate only if the exact machine/provider supports documented API/SDK/POS integration and merchant approval. First obtain brand, model and provider integration documentation. Until verified, use manual confirmation.

Order consistency: Re-check price and stock at submission; create order and line items safely; prevent duplicate orders on retry.

Notifications: Notify Admin of new orders and Customer of order/payment status changes. Database remains the source of truth; push notifications may be missed.

6. Product identification

Supported scan: QR and common barcode formats required by the client's actual product labels. Developer must test with sample labels.

Code mapping: Store unique scan code against product/SKU; prevent duplicate active codes.

Fallback: Unknown scan code must offer manual search and optionally a request-to-add-product action.

Important boundary: QR/barcode scanning is not AI recognition of a spare part from a normal photo. AI photo recognition is a separate feature.

7. Image compression, ZIP and storage

Image upload: Resize and compress images before cloud upload while keeping part details readable. Generate thumbnails if useful.

Cloud storage: Store optimized images by default. Keep original-quality copy only if Admin has a real need; avoid duplicate originals by default.

ZIP utility: Allow selecting supported files to create ZIP and selecting/downloading ZIP files to extract inside the app.

File safety: Validate type and size; prevent archive path traversal; limit extraction size/count; show progress, cancellation and errors.

Cleanup: Remove abandoned temporary local files. Define deletion rules for unused cloud images and backups.

Cost note: Compression can reduce image storage and bandwidth but does not remove database, backup or video costs. JPEG/MP4/ZIP may not compress much further.

8. O7A.pdf – source feature mapping

Main dashboard: PDF shows three entry cards/sections: Customer, Spare Parts and Service Center.

Customer screen: PDF lists Scanner, Video Assistants, Registration, Demo Video, Call + Visit, Transfer to Higher Department, Vehicle-related Information and Monitoring Options. It also shows a second Video Assistants entry.

Service Center screen: PDF lists Video Assistants, Demo Video, Spare Parts, Software Section Service and Book + Appointment.

Service Spare Parts: PDF shows Online purchase options and Offline purchase options.

Implementation: Registration, scanner, vehicle information, contact, spare-parts purchase and appointment request can be implemented as working flows. Demo/tutorial videos should use client-provided video files or links. Escalation can be a support request only after routing rules are approved.

Unclear terms: PDF does not define detailed behavior for Monitoring Options, Video Assistants, Transfer to Higher Department or Software Section Service. Do not invent logic. Record client clarification; use a basic placeholder/request flow only after approval.

Client content: Client must provide videos, phone numbers, service-centre details, visit address, service descriptions and escalation contacts.

9. Technical architecture requirements

Flutter structure: Organize by feature/module; separate UI, state/business logic, repositories and service integrations. Keep code maintainable and testable.

Backend choice: Choose one managed backend (e.g. Firebase or Supabase) after comparing authentication, database rules, file storage, backup/export, quotas and migration options. Avoid building two backends.

Server-side functions: Use managed serverless/edge functions for privileged work such as payment webhook verification, order validation and trusted notifications. Never embed secret keys in APK.

Ownership: Backend/cloud projects should be created under client ownership where possible. Developer gets least-privilege access and access is removable after handover.

Environments: Separate development/test from production. Do not run destructive tests on live data.

Network behavior: Use retry-safe operations and clear offline messaging. Do not show an order as submitted until backend confirms it.

10. Suggested data model

users: id, display_name, phone/email, role, created_at, account_status. Role changes restricted to trusted Admin processes.

vehicles: id, user_id, make, model, variant/year if needed.

categories: id, name, sort_order, is_active, created_at, updated_at.

products: id, name, sku/part_number, description, category_id, price, stock_qty, is_available, image_refs, compatible_vehicle_refs, timestamps.

product_scan_codes: id, product_id, code_value, code_type, is_active; enforce unique active code_value.

cart/cart_items: user_id, product_id, quantity, updated_at; decide local vs backend based on cross-device requirements.

orders: id, public_order_id, user_id, status, payment_status, subtotal, charges, total, delivery/contact snapshot, timestamps.

order_items: order_id, product_id reference, product_name/SKU/price snapshot, quantity, line_total; preserve order-time values.

payments: order_id, provider, provider_reference, amount, currency, status, verified_at, webhook_event_id.

appointments/support_requests: user_id, request_type, details, preferred_time if relevant, status, assigned_to, timestamps.

audit_logs: actor_user_id, action, entity_type/id, before/after summary, created_at.

11. Security and privacy

Authorization: Enforce customer/Admin permissions in database rules and privileged server functions. UI hiding is not security.

Customer isolation: Customer can access only their permitted profile, vehicles, cart, orders and support requests.

Secrets: Never place service-role keys, payment secrets or private API credentials in Flutter source/APK.

Payments: Verify provider webhooks server-side; validate signatures; prevent replay and duplicate processing.

Uploads: Restrict file type, size and storage paths. Protect private customer files.

Privacy: Use HTTPS; avoid logging passwords, OTPs or payment credentials; collect only necessary personal data; prepare privacy policy before Play Store release.

12. Notifications

Events: New order to Admin; order/payment status changes to Customer; appointment/support request to Admin; support response to Customer.

Privacy: Keep lock-screen text minimal; do not expose full addresses or sensitive data.

Reliability: When app opens from notification, refresh current data from backend.

WhatsApp: Tap-to-open contact/WhatsApp can be considered. Automated WhatsApp Business Cloud API is not assumed in MVP.

13. Testing plan

Functional: Registration; roles; product/category CRUD; compatibility; search; scanner success/no match/camera denied; cart; stock validation; order creation; duplicate retry; payment pending/verified/rejected; status changes; appointments/support.

Security: Customer cannot read another customer's data; non-admin cannot access Admin functions; forged/replayed webhook rejected; no secrets in APK; upload rules enforced.

Devices/network: Test multiple Android sizes, low-memory phone, slow/interrupted network, background/resume, expired session and retry after timeout.

Files: Test image quality/size, upload failure/retry, ZIP create/extract, corrupt ZIP, oversized archive, unsupported file and temporary-file cleanup.

Recovery: Demonstrate database export/backup and restore procedure.

14. Definition of Done / acceptance

Customer journey: Customer can register, browse, search/scan, inspect compatibility, add to cart and submit order.

Order: One successful submission creates one unique Order ID and correct order/item records; retry does not create duplicates.

Admin journey: Authorized Admin can manage products/stock, see orders and update order/payment states; changes persist after restart.

Tracking: Customer sees only their own orders and latest status.

Security: Backend denies unauthorized access even if a user modifies the app/UI.

Files: Images optimized before upload; ZIP utility safely validates and extracts supported archives.

Release: Critical/high severity bugs resolved; production config checked; release build generated; source and handover docs delivered.

15. Handover deliverables

Source: Complete Flutter source in client-owned repository or archive, with README and build instructions.

Builds: Testing APK and Play Store AAB as applicable.

Backend: Client-owned project, schema/rules/functions documentation, environment setup and backup/export instructions.

Accounts: List all third-party services, owner account, billing owner and access handover. Production ownership must not remain solely with developer.

Credentials: Transfer securely; do not put secrets in source or public docs. Rotate developer credentials after handover if needed.

Documents: Screen flow, data model, Admin guide, customer guide, known limitations, test report and release instructions.

Play Store: Prepare app listing, screenshots, icon, privacy policy and submission details. Google controls approval; developer cannot guarantee approval.

16. Client inputs needed before coding

Brand: App name, logo, colors and design references.

Catalogue: Initial products, categories, part numbers, prices, stock, photos and compatible scooter make/model list.

Business rules: Delivery/pickup, charges, minimum order, cancellation/refund rules, order statuses and approval responsibility. Unconfirmed policies must remain unpublished.

Payment: Merchant UPI QR/details, manual vs automatic confirmation preference, provider account/API/webhook access if available.

POS: Exact machine brand, model, provider and integration documents/contact.

Service: Service-centre locations, contact details, appointment rules.

Content: Demo/tutorial videos, support contacts, escalation destination and exact meaning of Monitoring Options/Video Assistants.

Store: Business details, privacy-policy information, app ownership and Play Console account.

17. Scope control and later roadmap

Not baseline unless separately approved: Website; iOS; AI photo recognition; live GPS/vehicle telemetry; technician live tracking; ERP/accounting; multi-warehouse sync; automated WhatsApp Cloud API; POS integration without provider API; advanced analytics.

Change request: Every new feature must have user story, acceptance criteria, data/security impact, external dependency and revised estimate before coding.

No silent assumptions: Where O7A.pdf or client has not defined behavior, developer must request written clarification and record approved behavior.

Priority sequence: MVP: login, catalogue, compatibility, search/scanner, cart, orders, Admin product/stock/order management, manual payment verification, basic notifications. Next: verified payment webhook and approved POS integration. Then: PDF service workflows, appointment, advanced support, optimization and analytics.

Working plan: Developer reviews this document, marks each requirement Must/Should/Later, confirms clarifications, produces screen designs and data schema, then breaks implementation into tasks and test cases.

Developer review / client approval

Developer name/company: _____

Reviewed version/date: _____

Clarifications attached: Yes / No

Client approval/signature: _____ Date: _____